



PARADIGMAS DE PROGRAMACIÓN REPRESENTATIVOS

Son los enfoques teóricos que definen la forma de estructurar y ejecutar el código. Se dividen en dos grandes familias:

IMPERATIVO (SE CENTRA EN CÓMO REALIZAR LAS TAREAS)

Describe la computación como una secuencia de instrucciones que cambian el estado del programa.

ESTRUCTURADO / PROCEDIMENTAL

- **Concepto:** Divide el código en funciones o procedimientos reutilizables. Evita saltos de código innecesarios (como GOTO).

- **Características:** Uso de estructuras de control (secuencia, selección if/else, iteración for/while).

- **Lenguajes representativos:** C, Pascal, BASIC.

ORIENTADO A OBJETOS (POO)

- **Concepto:** Organiza el código alrededor de "objetos" que combinan datos (atributos) y comportamientos (métodos).

- **Características:**
 1. Utiliza clases y objetos.
 2. Encapsula datos y métodos.
 3. Permite herencia.
 4. Utiliza polimorfismo.
 5. Utiliza abstracción.
 6. Facilita la reutilización y organización del código.

- **Lenguajes representativos:** Java, C++, C#, Python.

DECLARATIVO (SE CENTRA EN QUÉ RESULTADO SE DESEA OBTENER).

Describe qué debe resolver el programa sin detallar la secuencia exacta de pasos de control de flujo.

FUNCIONAL

- **Concepto:** Trata el cómputo como la evaluación de funciones matemáticas puras, evitando modificar el estado global o usar datos mutables.

- **Características:**
 1. Utiliza funciones como elementos principales.
 2. Favorece las funciones puras.
 3. Utiliza la recursividad.
 4. Favorece la inmutabilidad de los datos.
 5. Permite la composición de funciones.
 6. Reduce los efectos secundarios.
 7. Se concentra en qué resultado obtener.

- **Lenguajes representativos:** Haskell, Lisp, Elixir, Scala (y JS/Python en estilo funcional).

LÓGICO

- **Concepto:** Es un paradigma declarativo en el que un problema se representa mediante hechos, reglas y relaciones. El programa utiliza mecanismos de inferencia para encontrar respuestas a las consultas.

- **Características:**
 1. Utiliza hechos.
 2. Utiliza reglas lógicas.
 3. Permite realizar consultas.
 4. Utiliza inferencia lógica.
 5. El programador describe el problema en lugar de indicar todos los pasos para resolverlo.
 6. Es utilizado en sistemas expertos e inteligencia artificial.

- **Lenguajes representativos:** Prolog, Mercury, Datalog

PARADIGMAS DE PROGRAMACIÓN REPRESENTATIVOS

Son los enfoques teóricos que definen la forma de estructurar y ejecutar el código. Se dividen en dos grandes familias:



IMPERATIVO (SE CENTRA EN CÓMO REALIZAR LAS TAREAS)

Describe la computación como una secuencia de instrucciones que cambian el estado del programa.

PARADIGMA CONCURRENTE

- **Concepto:** Permite que varias tareas o procesos se ejecuten de manera concurrente, pudiendo trabajar al mismo tiempo o alternarse durante la ejecución

• Características:

1. Trabaja con múltiples procesos o tareas.
2. Permite la comunicación entre procesos.
3. Utiliza sincronización.
4. Puede utilizar hilos (threads).
5. Es útil para sistemas distribuidos y aplicaciones que realizan varias tareas.

• Lenguajes representativos:

Erlang, Go, Ada, Java

ORIENTADO A EVENTOS

- **Concepto:** El programa ejecuta determinadas acciones como respuesta a eventos, por ejemplo, hacer clic en un botón, presionar una tecla o recibir un mensaje.

• Características:

1. Se basa en eventos.
2. Utiliza manejadores de eventos.
3. Responde a acciones del usuario o del sistema.
4. Es común en interfaces gráficas.
5. También se utiliza en aplicaciones web.
6. El flujo del programa depende de los eventos que ocurren.

• Lenguajes representativos:

JavaScript, Visual Basic, C#, Java

DECLARATIVO (SE CENTRA EN QUÉ RESULTADO SE DESEA OBTENER).

Describe qué debe resolver el programa sin detallar la secuencia exacta de pasos de control de flujo.

PROGRAMACIÓN BASADA EN RESTRICCIONES

- **Concepto:** Es un paradigma declarativo en el que el problema se expresa mediante restricciones que deben cumplirse. El sistema busca valores que satisfagan todas las condiciones establecidas.

• Características:

1. Define restricciones sobre las variables.
2. Utiliza dominios de valores.
3. Busca soluciones que cumplan las restricciones.
4. Es útil para problemas de planificación y asignación.
5. El programador especifica las condiciones que debe cumplir la solución.

• Lenguajes representativos:

Prolog con restricciones (CLP), MiniZinc, Oz